

Documentazione Test

1. Test di Unità

Test eseguiti in isolamento, senza accesso a file system, database o socket di rete

Package: `data`

Classe Testata: `ContinuousAttribute`

Test 1 (`testGetName`)

Scopo del Test: verifica l'aderenza al principio di riuso tramite ereditarietà: l'attributo numerico eredita correttamente il comportamento di `getName()` dalla superclasse comune, restituendo il nome simbolico assegnato senza duplicazione di logica.

Campo	Contenuto
Input	<code>name = "chilometri"</code> passato al costruttore
Azione	<code>ca.getName()</code> (metodo ereditato)
Atteso	restituisce esattamente <code>"chilometri"</code>

Test 2 (`testGetIndex`)

Scopo del Test: verifica che l'incapsulamento dello stato interno preservi correttamente l'indice identificativo dell'attributo, memorizzato e restituito in coerenza con il valore fornito in costruzione.

Campo	Contenuto
Input	<code>index = 2</code> passato al costruttore
Azione	<code>ca.getIndex()</code>
Atteso	restituisce <code>2</code>

Classe Testata: `DiscreteAttribute`

Test 1 (`testNameAndIndex`)

Scopo del Test: verifica che il principio di ereditarietà applicato alla gerarchia `Attribute` garantisca la corretta inizializzazione di nome e indice anche per la sottoclasse discreta.

Campo	Contenuto
Input	<code>name</code> e <code>index</code> passati al costruttore
Azione	<code>da.getName()</code> e <code>da.getIndex()</code>
Atteso	entrambi i metodi restituiscono i valori passati al costruttore

Test 2 (`testNumberOfDistinctValues`)

Scopo del Test: verifica che il numero di valori distinti gestiti dall'attributo discreto rifletta fedelmente gli elementi inseriti, a conferma della corretta integrità della struttura dati sottostante.

Campo	Contenuto
Input	insieme di valori discreti inseriti nel costruttore
Azione	<code>da.getNumberOfDistinctValue()</code>
Atteso	restituisce il numero esatto di elementi inseriti

Test 3 (`testEliminazioneValoriDuplicati`)

Scopo del Test: verifica una precisa scelta architetturale: l'adozione di `TreeSet` come struttura dati interna, che garantisce per costruzione l'unicità dei valori ed elimina automaticamente i duplicati, assicurando che la cardinalità riportata rifletta solo elementi realmente distinti.

Campo	Contenuto
Input	array di valori con ripetizioni, es. <code>["Fiat", "Fiat", "Ford"]</code>
Azione	costruzione dell'attributo con l'array fornito, poi <code>da.getNumberOfDistinctValue()</code>
Atteso	il conteggio restituito è <code>2</code> (i doppioni non vengono considerati)

Test 4 (`testIteratorReturnsValues`)

Scopo del Test: verifica la corretta implementazione del contratto dell'interfaccia `Iterable`, principio architetturale che consente di attraversare in modo uniforme e polimorfico tutti i valori memorizzati nell'attributo.

Campo	Contenuto
Input	attributo discreto popolato con un insieme noto di valori
Azione	ciclo <code>for-each</code> su <code>da</code>
Atteso	l'iteratore attraversa tutti i valori memorizzati, senza omissioni

Classe Testata: `Attribute`

Test 1 (`testGetName`)

Scopo del Test: verifica il principio di incapsulamento alla base della classe: la corretta memorizzazione e restituzione del nome simbolico dell'attributo base.

Campo	Contenuto
Input	<code>name = "temperatura"</code> passato al costruttore
Azione	<code>a.getName()</code>
Atteso	restituisce esattamente <code>"temperatura"</code>

Test 2 (`testGetIndex`)

Scopo del Test: verifica il principio di incapsulamento applicato all'identificativo numerico posizionale dell'attributo, correttamente memorizzato e restituito.

Campo	Contenuto
Input	<code>index = 5</code> passato al costruttore
Azione	<code>a.getIndex()</code>
Atteso	restituisce <code>5</code>

Classe Testata: `TrainingDataException`

Test 1 (`testExceptionMessage`)

Scopo del Test: verifica l'aderenza al principio architetturale di propagazione controllata degli errori: il messaggio diagnostico viene correttamente trasmesso attraverso la gerarchia delle eccezioni, garantendo la tracciabilità dell'errore.

Campo	Contenuto
Input	stringa diagnostica di prova passata al costruttore
Azione	<code>new TrainingDataException(messaggio)</code> seguito da <code>getMessage()</code> (metodo ereditato)
Atteso	<code>getMessage()</code> restituisce esattamente il testo fornito al costruttore

Package: database

Classe Testata: Column

Test 1 (`testGetColumnName`)

Scopo del Test: verifica il principio di incapsulamento: il nome della colonna viene correttamente memorizzato e restituito.

Campo	Contenuto
Input	nome colonna passato al costruttore
Azione	<code>col.getColumnName()</code>
Atteso	restituisce esattamente il valore assegnato tramite costruttore

Test 2 (`testIsNumberTrue` e `testIsNumberFalse`)

Scopo del Test: verifica la corretta implementazione del meccanismo di riconoscimento del tipo di dato, elemento architetturale alla base della successiva mappatura tra colonne e attributi discreti/continui.

Campo	Contenuto
Input	colonna con tipo <code>"number"</code> (caso <i>true</i>); colonna con tipo <code>"string"</code> (caso <i>false</i>)
Azione	<code>col.isNumber()</code>

Atteso	restituisce <code>true</code> per il tipo <code>"number"</code> ; restituisce <code>false</code> per il tipo <code>"string"</code>
--------	--

Test 3 (`testUnknownTypeIsNotNumber`)

Scopo del Test: verifica a fronte di input non validi: un tipo di colonna non riconosciuto non deve essere erroneamente classificato come numerico, a garanzia della robustezza del meccanismo di tipizzazione.

Campo	Contenuto
Input	colonna con tipo generico non standard, es. <code>"date"</code>
Azione	<code>col.isNumber()</code>
Atteso	restituisce <code>false</code>

Test 4 (`testToString`)

Scopo del Test: verifica che la rappresentazione testuale della colonna segua il formato standard previsto, in coerenza con il contratto di rappresentazione adottato nel progetto.

Campo	Contenuto
Input	colonna con nome e tipo valorizzati
Azione	<code>col.toString()</code>
Atteso	restituisce la stringa formattata secondo lo schema <code>nome:tipo</code>

Classe Testata: `Example`

Test 1 (`testAddAndGet`)

Scopo del Test: verifica il principio architetturale di preservazione dell'ordine e del tipo: l'ordine di inserimento degli elementi della tupla viene rispettato e ciascun elemento è recuperabile all'indice corretto, con il tipo originario preservato.

Campo	Contenuto
Input	una stringa e un valore a virgola mobile inseriti in sequenza
Azione	<code>ex.add(...)</code> per ciascun elemento, poi <code>ex.get(int)</code> per ogni indice

Atteso	<code>get(i)</code> restituisce l'elemento atteso all'indice <code>i</code> , con il tipo originario preservato
--------	---

Test 2 (`testGetOutOfBounds`)

Scopo del Test: verifica a fronte di input non validi: l'accesso a un indice non presente nella tupla viene correttamente intercettato, rispettando i limiti della lista interna.

Campo	Contenuto
Input	indice non valido (fuori dai limiti della lista interna)
Azione	<code>ex.get(indiceNonValido)</code>
Atteso	viene sollevata <code>IndexOutOfBoundsException</code>

Test 3 (`testToString`)

Scopo del Test: verifica che la serializzazione testuale della tupla produca una rappresentazione leggibile, coerente con il contratto architetturale adottato per la rappresentazione degli oggetti di dominio.

Campo	Contenuto
Input	tupla <code>ex</code> popolata con più valori
Azione	<code>ex.toString()</code>
Atteso	restituisce una sequenza di elementi separati da spazio, contenente tutti i valori inseriti

Test 4 (`testCompareToEqual` e `testCompareToDifferent`)

Scopo del Test: verifica la corretta implementazione del contratto dell'interfaccia `Comparable` , principio architetturale che garantisce coerenza nell'ordinamento: tuple identiche risultano equivalenti, tuple con almeno un valore differente no.

Campo	Contenuto
Input	due tuple identiche (caso <i>equal</i>); due tuple che differiscono in una posizione (caso <i>different</i>)
Azione	<code>ex1.compareTo(ex2)</code>

Atteso	restituisce <code>0</code> per le tuple identiche; restituisce un valore diverso da zero quando è presente una differenza
--------	---

Test 5 (`testIterator`)

Scopo del Test: verifica il rispetto del contratto dell'interfaccia `Iterable` , che consente di scorrere in modo uniforme tutti gli elementi registrati nella tupla.

Campo	Contenuto
Input	tupla <code>ex</code> popolata con un insieme noto di elementi
Azione	ciclo <code>for-each</code> su <code>ex</code>
Atteso	tutti gli elementi registrati vengono considerati correttamente, senza omissioni

Classe Testata: `DatabaseConnectionException`

Test 1 (`testMessage`)

Scopo del Test: verifica il principio di incapsulamento del messaggio diagnostico di errore, correttamente memorizzato e restituito.

Campo	Contenuto
Input	messaggio descrittivo passato al costruttore
Azione	<code>new DatabaseConnectionException(messaggio)</code> seguito da <code>getMessage()</code>
Atteso	<code>getMessage()</code> restituisce esattamente la stringa fornita

Test 2 (`testIsException`)

Scopo del Test: verifica che la classe sia correttamente collocata nella gerarchia standard delle eccezioni Java, principio architetturale che la qualifica come *checked exception*.

Campo	Contenuto
Input	istanza di <code>DatabaseConnectionException</code>
Azione	asserzione di tipo (<code>instanceof</code>)
Atteso	l'istanza è riconosciuta come sottoclasse di <code>java.lang.Exception</code>

Classe Testata: `EmptySetException`

Test 1 (`testMessage`)

Scopo del Test: verifica che l'eccezione, istanziata senza argomenti, esponga un messaggio diagnostico standard e predefinito, a garanzia della coerenza del contratto di errore.

Campo	Contenuto
Input	nessuno (costruttore senza argomenti)
Azione	<code>new EmptySetException()</code> seguito da <code>getMessage()</code>
Atteso	<code>getMessage()</code> restituisce esattamente la stringa standard <code>"[!] The result set is empty"</code>

Test 2 (`testIsException`)

Scopo del Test: verifica che la classe erediti correttamente da `java.lang.Exception`, principio architetturale che ne garantisce il comportamento come *checked exception*.

Campo	Contenuto
Input	istanza di <code>EmptySetException</code>
Azione	asserzione di tipo (<code>instanceof</code>)
Atteso	l'istanza è riconosciuta come sottoclasse di <code>java.lang.Exception</code>

Package: `server` (continua)

Classe Testata: `UnknownValueException`

Test 1 (`testMessage`)

Scopo del Test: verifica la corretta inizializzazione e conservazione del messaggio di errore diagnostico, in coerenza con il principio di incapsulamento adottato nella gerarchia delle eccezioni.

Campo	Contenuto
Input	stringa specifica passata al costruttore

Azione	<code>new UnknownValueException(messaggio)</code> seguito da <code>getMessage()</code>
Atteso	<code>getMessage()</code> restituisce esattamente il testo fornito originariamente

Test 2 (`testIsException`)

Scopo del Test: verifica che la classe estenda direttamente `java.lang.Exception`, principio architetturale che la qualifica come *checked exception* e obbliga i chiamanti a gestirne esplicitamente la cattura.

Campo	Contenuto
Input	istanza di <code>UnknownValueException</code>
Azione	asserzione di tipo (<code>instanceof</code>)
Atteso	l'istanza è riconosciuta come sottoclasse diretta di <code>java.lang.Exception</code>

2. Test di Integrazione

Test che verificano l'interazione tra i moduli interni o con sistemi esterni (es. database MySQL).

Package: `tree`

Classe Testata: `DiscreteNode`

Test 1 (`testGenerazioneRami`)

Scopo del Test: verifica il principio architetturale di generazione dinamica della struttura: il nodo genera automaticamente un ramo per ciascun valore distinto assunto dall'attributo discreto su cui è basato lo split.

Campo	Contenuto
Input	dati della colonna discreta <code>X</code> , con valori distinti <code>"A"</code> e <code>"B"</code>
Azione	costruzione di <code>DiscreteNode</code> sui dati forniti
Atteso	vengono generati esattamente <code>2</code> rami distinti (uno per ciascun valore)

Test 2 (`testCondizioneValoreEsistente`)

Scopo del Test: verifica che, dato un valore di input noto, il nodo individui correttamente il ramo di split corrispondente, a conferma della coerenza del meccanismo di instradamento.

Campo	Contenuto
Input	valore esistente "A"
Azione	<code>node.testCondition("A")</code>
Atteso	restituisce un indice valido (0 o 1) corrispondente al ramo generato per quel valore

Test 3 (`testCondizioneValoreInesistente`)

Scopo del Test: verifica a fronte di input non validi: un valore non previsto nel training set originale viene gestito senza produrre corrispondenze errate, a garanzia della robustezza architetturale del nodo.

Campo	Contenuto
Input	valore testuale non mappato, es. "Z"
Azione	<code>node.testCondition("Z")</code>
Atteso	restituisce -1

Classe Testata: `ContinuousNode`

Test 1 (`testGenerazioneRami`)

Scopo del Test: verifica che l'algoritmo di minimizzazione della varianza, principio cardine della strategia di split adottata, identifichi correttamente una soglia valida per un attributo continuo, generando i rami figli conseguenti.

Campo	Contenuto
Input	dati numerici della colonna Y
Azione	costruzione di <code>ContinuousNode</code> sui dati forniti
Atteso	viene identificata una soglia di split valida; vengono generati almeno un ramo figlio

Test 2 (`testValutazioneSoglia`)

Scopo del Test: verifica che un valore continuo in input venga instradato correttamente attraverso gli operatori relazionali definiti dalla soglia di split del nodo, elemento chiave del design del meccanismo di decisione.

Campo	Contenuto
Input	valore numerico di input, es. <code>1.0</code>
Azione	<code>node.testCondition(1.0)</code>
Atteso	l'indice restituito identifica coerentemente il ramo di partizione corretto, oppure <code>-1</code> in assenza di corrispondenza

Classe Testata: `LeafNode`

Test 1 (`testCalcoloValorePredetto`)

Scopo del Test: verifica la correttezza aritmetica del calcolo della media dei valori target assegnati alla foglia, componente centrale del principio di predizione basato su aggregazione statistica.

Campo	Contenuto
Input	insieme di esempi (<code>Example</code>) assegnati alla foglia, con valori target numerici (indici <code>[beginExampleIndex, endExampleIndex]</code>)
Azione	costruzione di <code>LeafNode</code> sugli esempi forniti, poi <code>leaf.getPredictedClassValue()</code>
Atteso	il valore restituito è un numero in virgola mobile formalmente valido (<code>!Double.isNaN(valore)</code>)

Test 2 (`testNumeroRamiFigli`)

Scopo del Test: verifica il rispetto del principio architetturale che definisce il nodo foglia come elemento terminale della gerarchia: una foglia non può possedere rami discendenti.

Campo	Contenuto
Input	istanza di <code>LeafNode</code>
Azione	<code>leaf.getNumberOfChildren()</code>

Atteso	restituisce 0
--------	---------------

Test 3 (testToString)

Scopo del Test: verifica che la rappresentazione testuale del nodo terminale contenga l'indicazione esplicita che si tratta di una foglia, in coerenza con il contratto di rappresentazione condiviso da tutta la gerarchia `Node` .

Campo	Contenuto
Input	istanza di <code>LeafNode</code>
Azione	<code>leaf.toString()</code>
Atteso	la stringa generata contiene esplicitamente la parola chiave <code>"LEAF"</code>

Classe Testata: `Node` (testata tramite l'estensione concreta `LeafNode`)

Test 1 (testInizializzazioneIndici)

Scopo del Test: verifica il principio di incapsulamento dello stato comune della gerarchia: gli estremi dell'intervallo di esempi coperti dal nodo vengono correttamente assegnati e recuperabili.

Campo	Contenuto
Input	<code>trainingSet</code> , <code>beginExampleIndex</code> , <code>endExampleIndex</code> passati al costruttore di <code>Node</code> (tramite <code>LeafNode</code>)
Azione	<code>node.getBeginExampleIndex()</code> e <code>node.getEndExampleIndex()</code>
Atteso	i due metodi restituiscono esattamente <code>beginExampleIndex</code> ed <code>endExampleIndex</code> passati al costruttore

Test 2 (testGenerazioneIdProgressivo)

Scopo del Test: verifica il principio architetturale di identificazione univoca tramite contatore condiviso: l'identificativo del nodo viene generato in modo progressivo, garantendo unicità crescente tra nodi creati in sequenza.

Campo	Contenuto
Input	un primo nodo creato nel setup (es. <code>LeafNode</code>); un secondo nodo creato successivamente
Azione	confronto tra <code>secondoNodo.getIdNode()</code> e <code>primoNodo.getIdNode()</code> (il contatore <code>idNodeCount</code> viene incrementato ad ogni costruzione)
Atteso	<code>secondoNodo.getIdNode() > primoNodo.getIdNode()</code>

Test 3 (`testCalcoloVarianza`)

Scopo del Test: verifica la coerenza matematica del calcolo della varianza effettuato nel costruttore del nodo, elemento architetturale alla base della strategia di split del regression tree.

Campo	Contenuto
Input	<code>trainingSet</code> con esempi coperti dal nodo (intervallo <code>[beginExampleIndex, endExampleIndex]</code>)
Azione	costruzione del nodo (calcolo interno dell'SSE rispetto alla media dei <code>getClassValue()</code>), poi <code>node.getVariance()</code>
Atteso	<code>node.getVariance() >= 0.0</code>

Test 4 (`testToString`)

Scopo del Test: verifica che la stringa descrittiva del nodo riporti i riferimenti chiave necessari alla diagnostica, in coerenza con il contratto di rappresentazione condiviso dalla gerarchia.

Campo	Contenuto
Input	nodo con <code>beginExampleIndex</code> , <code>endExampleIndex</code> e <code>variance</code> già calcolati
Azione	<code>node.toString()</code>
Atteso	la stringa restituita è nel formato esatto <code>"Nodo: [Examples:<begin>-<end>] variance:<variance>"</code>

Classe Testata: `SplitNode` (testata mediante l'implementazione concreta `DiscreteNode`)

Test 1 (`testGetAttribute`)

Scopo del Test: verifica il principio di incapsulamento dell'attributo indipendente su cui è generato lo split, assicurando che sia recuperabile fedelmente.

Campo	Contenuto
Input	istanza di <code>Attribute</code> (es. <code>DiscreteAttribute</code>) passata al costruttore di <code>DiscreteNode</code> / <code>ContinuousNode</code>
Azione	<code>splitNode.getAttribute()</code>
Atteso	restituisce esattamente l'istanza di <code>Attribute</code> passata alla costruzione (campo privato <code>attribute</code>)

Test 2 (`testCalcoloSplitVariance`)

Scopo del Test: verifica la validità matematica della varianza complessiva di split, calcolata come somma delle varianze dei rami figli, principio architetturale alla base della strategia ricorsiva di costruzione dell'albero.

Campo	Contenuto
Input	sotto-insieme di dati di training <code>[beginExampleIndex, endExampleIndex]</code> usato per generare i rami figli
Azione	costruzione del nodo (il costruttore di <code>SplitNode</code> ordina il training set, invoca <code>setSplitInfo()</code> e somma, per ogni elemento di <code>mapSplit</code> , la varianza di un <code>LeafNode</code> temporaneo costruito su quel ramo), poi <code>splitNode.getVariance()</code>
Atteso	<code>splitNode.getVariance()</code> (la <code>splitVariance</code>) restituisce un numero reale <code>>= 0.0</code>

Test 3 (`testGestioneFigliESplitInfo`)

Scopo del Test: verifica il corretto popolamento della struttura dati dei rami figli (`mapSplit`), elemento architetturale centrale nella modellazione della gerarchia dei nodi, e l'accessibilità dei relativi oggetti aggregati `SplitInfo`.

Campo	Contenuto
Input	nodo di split con rami figli già generati (popolati nella lista protetta <code>mapSplit</code>)
Azione	<code>splitNode.getNumberOfChildren()</code> (restituisce <code>mapSplit.size()</code>) e <code>splitNode.getSplitInfo(0)</code> (restituisce <code>mapSplit.get(0)</code>)
Atteso	<code>getNumberOfChildren()</code> è <code>> 0</code> e conferma la presenza dei rami; <code>getSplitInfo(0)</code> restituisce un'istanza non nulla di <code>SplitNode.SplitInfo</code>

Test 4 (`testFormulateQuery`)

Scopo del Test: verifica che il nodo sappia esportare in forma testuale le condizioni di test associate ai propri rami, in coerenza con il principio di disaccoppiamento che consente la comunicazione delle regole di split verso il server tramite un formato testuale condiviso.

Campo	Contenuto
Input	nodo di split con rami figli configurati in <code>mapSplit</code>
Azione	<code>splitNode.formulateQuery()</code>
Atteso	la stringa risultante concatena, per ciascun ramo <code>i</code> , una riga nel formato <code>"<i>:<nomeAttributo><comparator><splitValue>\n"</code> , includendo quindi sia il nome dell'attributo di split sia il comparatore e il valore di ciascun ramo

Test 5 (`testCompareTo`)

Scopo del Test: verifica la corretta implementazione del contratto dell'interfaccia `Comparable` , assicurando che il confronto tra nodi con varianze equivalenti sia coerente.

Campo	Contenuto
Input	un secondo nodo con caratteristiche e dati analoghi al primo, tali da produrre <code>getVariance()</code> equivalente
Azione	<code>nodo1.compareTo(nodo2)</code>
Atteso	restituisce <code>0</code> quando <code>nodo1.getVariance() == nodo2.getVariance()</code> ; restituisce <code>-1</code> se <code>nodo1.getVariance() < nodo2.getVariance()</code> ; restituisce <code>1</code> se <code>nodo1.getVariance() > nodo2.getVariance()</code>

Classe Testata: RegressionTree

Test 1 (testInduzioneAlbero)

Scopo del Test: verifica il principio architetturale di composizione ricorsiva: la corretta costruzione strutturale dell'intero albero di regressione a partire dal dataset di training.

Campo	Contenuto
Input	Data trainingSet caricato dalla tabella provaC
Azione	new RegressionTree(trainingSet) (invoca internamente learnTree() , che ricorsivamente crea LeafNode o SplitNode in base alla soglia numberOfExamplesPerLeaf = 10% degli esempi)
Atteso	il costruttore istanzia correttamente la gerarchia dei nodi (root e, se non foglia, childTree[]); tree.toString() non è null e contiene la stringa "SPLIT" (da SplitNode.toString()) e/o "LEAF" (da LeafNode.toString())

Test 2 (testComportamentoNodoRadice)

Scopo del Test: verifica il comportamento polimorfico dei metodi di estrazione dati quando invocati sul nodo radice, distinguendo coerentemente il caso di nodo decisionale da quello di nodo foglia.

Campo	Contenuto
Input	albero generato su provaC , la cui radice è un nodo decisionale (non terminale)
Azione	tree.getCurrentNodeQuery() e tree.getPredictedValue()
Atteso	getCurrentNodeQuery() restituisce una stringa valida (la condizione di split); getPredictedValue() restituisce null (comportamento atteso per un nodo non foglia)

Test 3 (testNavigazioneEccezione)

Scopo del Test: verifica a fronte di input non validi: l'accesso a un ramo figlio inesistente durante la navigazione dell'albero viene gestito sollevando l'eccezione personalizzata UnknownValueException .

Campo	Contenuto
-------	-----------

Input	indice non valido, es. <code>-1</code>
Azione	<code>tree.getChild(-1)</code>
Atteso	viene sollevata l'eccezione personalizzata <code>UnknownValueException</code>

Test 4 (`testSerializzazione`)

Scopo del Test: verifica l'aderenza al contratto dell'interfaccia `Serializable`, principio architetturale che garantisce la persistenza dell'albero su file e il suo corretto ripristino mantenendo intatta la struttura originaria.

Campo	Contenuto
Input	istanza di <code>RegressionTree</code> già costruita; percorso di un file temporaneo locale (<code>"test_tree.ser"</code>)
Azione	<code>tree.salva("test_tree.ser")</code> (serializza l'oggetto con <code>ObjectOutputStream</code>), seguito da <code>RegressionTree.carica("test_tree.ser")</code> (metodo statico, deserializza con <code>ObjectInputStream</code>)
Atteso	il file <code>"test_tree.ser"</code> viene effettivamente creato; l'albero restituito da <code>carica()</code> produce lo stesso output di <code>toString()</code> dell'albero originale

Package: `data`

Classe Testata: `Data`

Test 1 (`tabellaInesistente_throwsTrainingDataException`)

Scopo del Test: verifica a fronte di input non validi: l'accesso a una tabella non esistente viene gestito in modo controllato, sollevando un'eccezione applicativa in coerenza con il principio architetturale di gestione esplicita degli errori.

Campo	Contenuto
Input	nome di una tabella non esistente nel database
Azione	<code>new Data(nomeTabellaInesistente)</code>
Atteso	viene sollevata <code>TrainingDataException</code>

Test 2 (`tabellaConSingolaColonna_throwsTrainingDataException`)

Scopo del Test: verifica a fronte di input non validi: la validazione strutturale dello schema della tabella assicura che sia presente almeno un attributo esplicativo e uno target, principio di integrità del dominio applicativo.

Campo	Contenuto
Input	tabella con meno di due colonne
Azione	<code>new Data(nomeTabella)</code>
Atteso	viene sollevata <code>TrainingDataException</code>

Test 3 (`targetNonNumerico_throwsTrainingDataException`)

Scopo del Test: verifica a fronte di input non validi: il vincolo di dominio sull'attributo di classe, richiesto obbligatoriamente per gli alberi di regressione, viene applicato correttamente in fase di caricamento.

Campo	Contenuto
Input	tabella la cui ultima colonna (target) non è di tipo numerico continuo
Azione	<code>new Data(nomeTabella)</code>
Atteso	viene sollevata <code>TrainingDataException</code>

Test 4 (`identificaTipoColonnaCorrettamente`)

Scopo del Test: verifica il meccanismo architetturale di mappatura dinamica dei tipi sulle colonne della tabella caricata, distinguendo correttamente attributi discreti e continui.

Campo	Contenuto
Input	dataset caricato dalla tabella <code>provaC</code> (colonna testuale <code>X</code> all'indice 0, colonna numerica <code>Y</code> all'indice 1)
Azione	<code>data.getExplanatoryAttribute(0)</code> e <code>data.getExplanatoryAttribute(1)</code> (gli attributi esplicativi sono popolati nel costruttore in base a <code>Column.isNumber()</code>)

Atteso	<code>getExplanatoryAttribute(0)</code> restituisce un'istanza di <code>DiscreteAttribute</code> (colonna X); <code>getExplanatoryAttribute(1)</code> restituisce un'istanza di <code>ContinuousAttribute</code> (colonna Y)
--------	--

Test 5 (`getExplanatoryValue_ritornaValoreNonNullo` e `_throwsIndexOutOfBoundsException`)

Scopo del Test: verifica le garanzie di affidabilità della matrice degli esempi caricati e, a fronte di input non validi, il rispetto dei limiti di accesso agli indici.

Campo	Contenuto
Input	indici (riga, colonna) validi (caso <i>non nullo</i>); indici fuori range (caso <i>eccezione</i>)
Azione	<code>data.getExplanatoryValue(riga, colonna)</code>
Atteso	con indici validi restituisce un oggetto non nullo; con indici fuori range viene sollevata <code>IndexOutOfBoundsException</code>

Test 6 (`getClassAttribute_attributoTargetCorretto` e `getClassValue_ritornaIndiceValido`)

Scopo del Test: verifica il corretto binding tra la variabile target del dataset e la sua rappresentazione interna, principio architetturale di separazione tra attributi esplicativi e attributo di classe.

Campo	Contenuto
Input	dataset caricato con colonna target <code>C</code> come ultima colonna
Azione	<code>data.getClassAttribute()</code> (metodo <i>package-private</i> , quindi il test deve trovarsi nel package <code>data</code>) e <code>data.getClassValue(riga)</code>
Atteso	<code>getClassAttribute()</code> restituisce l'oggetto <code>ContinuousAttribute</code> corrispondente all'ultima colonna (nome <code>"C"</code> e indice <code>numAttributes - 1</code>); <code>getClassValue(riga)</code> restituisce il valore <code>double</code> primitivo del target per quella riga, senza sollevare errori

Test 7 (`ordinamentoCorretto_discreteAttribute`)

Scopo del Test: verifica l'implementazione dell'algoritmo di ordinamento interno (Quicksort), componente architetturale condivisa, applicato ai valori testuali di un attributo

discreto.

Campo	Contenuto
Input	dataset con colonna discreta <code>X</code> ; intervallo completo <code>[0, data.getNumberOfExamples() - 1]</code>
Azione	<code>data.sort(attributoX, 0, data.getNumberOfExamples() - 1)</code> (internamente invoca <code>quicksort()</code> , che per gli attributi discreti usa <code>String.compareTo()</code> come criterio di partizionamento)
Atteso	dopo l'ordinamento, iterando sulle righe tramite <code>getExplanatoryValue(i, indiceX)</code> , i valori della colonna <code>X</code> risultano in ordine alfabetico crescente

Test 8 (`ordinamentoCorretto_continuousAttribute`)

Scopo del Test: verifica l'implementazione dell'algoritmo di ordinamento interno (Quicksort), componente architetturale condivisa, applicato ai valori reali di un attributo continuo.

Campo	Contenuto
Input	dataset con colonna continua <code>Y</code> ; intervallo completo <code>[0, data.getNumberOfExamples() - 1]</code>
Azione	<code>data.sort(attributoY, 0, data.getNumberOfExamples() - 1)</code> (internamente invoca <code>quicksort()</code> , che per gli attributi continui usa <code>Double.compareTo()</code> come criterio di partizionamento)
Atteso	dopo l'ordinamento, iterando con <code>getExplanatoryValue(i, indiceY)</code> , tutti gli elementi della colonna <code>Y</code> risultano in ordine numerico crescente

Package: `database`

Classe Testata: `TableData`

Test 1 (`testGetTransazioniValide`)

Scopo del Test: verifica che l'esecuzione della query `SELECT` produca correttamente una lista ordinata di tuple `Example` , in coerenza con il principio architetturale di mappatura tra risultati del database e oggetti di dominio.

Campo	Contenuto
Input	tabella "provaC"
Azione	<code>tableData.getTransazioni("provaC")</code>
Atteso	la <code>List<Example></code> restituita non è <code>null</code> né vuota

Test 2 (`testGetTransazioniTabellaInesistente`)

Scopo del Test: verifica a fronte di input non validi: l'estrazione da una tabella non esistente viene gestita sollevando esplicitamente `SQLException`, a garanzia della robustezza architetturale dell'accesso ai dati.

Campo	Contenuto
Input	nome di una tabella inesistente
Azione	<code>tableData.getTransazioni(nomeTabellaInesistente)</code>
Atteso	viene sollevata <code>SQLException</code> ; nel codice questo avviene perché <code>TableSchema</code> per una tabella inesistente produce <code>getNumberOfAttributes() == 0</code> , condizione per cui <code>getTransazioni()</code> solleva esplicitamente <code>new SQLException()</code> prima ancora di eseguire la query

Test 3 (`testGetDistinctColumnValuesString`)

Scopo del Test: verifica la scelta architetturale di adottare `TreeSet` per l'estrazione e l'ordinamento dei valori unici di tipo stringa presenti in una colonna.

Campo	Contenuto
Input	tabella "provaC"; oggetto <code>Column</code> per la colonna testuale X
Azione	<code>tableData.getDistinctColumnValues("provaC", columnX)</code> (esegue <code>SELECT DISTINCT X FROM provaC ORDER BY X ASC</code>)
Atteso	il <code>Set<Object></code> (basato su <code>TreeSet</code>) restituito non è vuoto e contiene valori di tipo <code>String</code>

Test 4 (`testGetDistinctColumnValuesNumber`)

Scopo del Test: verifica la scelta architetturale di adottare `TreeSet` per l'estrazione e l'ordinamento dei valori unici di tipo numerico (`Double`) presenti in una colonna.

Campo	Contenuto
Input	tabella <code>"provaC"</code> ; oggetto <code>Column</code> per la colonna numerica <code>Y</code>
Azione	<code>tableData.getDistinctColumnValues("provaC", columnY)</code> (poiché <code>columnY.isNumber() == true</code> , i valori vengono letti con <code>rs.getDouble(1)</code>)
Atteso	il <code>Set<Object></code> restituito non è vuoto e contiene valori di tipo <code>Double</code>

Classe Testata: `TableSchema`

Test 1 (`testGetNumberOfAttributes`)

Scopo del Test: verifica l'estrazione e il conteggio corretto delle colonne dal dizionario dei dati del database, elemento architetturale alla base della costruzione dello schema.

Campo	Contenuto
Input	<code>db</code> connesso; tabella esistente nel database, es. <code>"provaC"</code>
Azione	<code>new TableSchema(db, "provaC")</code> seguito da <code>schema.getNumberOfAttributes()</code>
Atteso	restituisce un valore strettamente maggiore di zero

Test 2 (`testGetColumn` e `testGetColumnOutOfBounds`)

Scopo del Test: verifica l'accesso posizionale alla definizione dei tipi della singola colonna e, a fronte di input non validi, la corretta gestione di richieste con indici non presenti nello schema.

Campo	Contenuto
Input	indice valido (caso <i>getColumn</i>); indice non presente nello schema (caso <i>out of bounds</i>)
Azione	<code>schema.getColumn(indice)</code>

Atteso	con indice valido restituisce l'oggetto <code>Column</code> corrispondente; con indice non valido viene sollevata <code>IndexOutOfBoundsException</code>
--------	--

Test 3 (`testIterator`)

Scopo del Test: verifica il rispetto del contratto dell'interfaccia `Iterable` , che consente di scorrere sequenzialmente tutte le colonne dello schema.

Campo	Contenuto
Input	schema popolato con un insieme noto di colonne
Azione	ciclo <code>for-each</code> su <code>schema</code>
Atteso	il numero di cicli eseguiti corrisponde esattamente alla cardinalità degli attributi

Test 4 (`testTabellaInesistente`)

Scopo del Test: verifica a fronte di input non validi: il costruttore gestisce in modo robusto la scansione dei metadati per una tabella non definita nel database, in coerenza con il principio architetturale di tolleranza agli errori sui dati esterni.

Campo	Contenuto
Input	nome di una tabella fittizia (non esistente)
Azione	<code>new TableSchema(nomeTabellaFittizia)</code>
Atteso	l'input non valido viene gestito senza propagare errori non controllati: il <code>ResultSet</code> viene semplicemente esaurito e la lista interna delle colonne resta vuota (cardinalità pari a <code>0</code>)

Classe Testata: `DbAccess`

Test 1 (`testInitAndGetConnection`)

Scopo del Test: verifica l'inizializzazione della connessione al database MySQL, principio architetturale di gestione centralizzata della sessione, e il corretto recupero della connessione attiva.

Campo	Contenuto
-------	-----------

Input	parametri di rete predefiniti hard-coded nella classe (<code>SERVER = "localhost"</code> , <code>PORT = 3306</code> , <code>DATABASE = "MapDB"</code> , <code>USER_ID = "MapUser"</code>)
Azione	<code>dbAccess.initConnection()</code> (carica il driver <code>com.mysql.cj.jdbc.Driver</code> e invoca <code>DriverManager.getConnection()</code>) seguito da <code>dbAccess.getConnection()</code> (metodo <i>package-private</i> , quindi il test deve trovarsi nel package <code>database</code>)
Atteso	l'oggetto <code>Connection</code> restituito non è <code>null</code> ; <code>connection.isClosed() == false</code> ; non viene sollevata <code>DatabaseConnectionException</code>

Test 2 (`testCloseConnection`)

Scopo del Test: verifica la corretta chiusura e il rilascio delle risorse di rete allocate per la sessione, in coerenza con il principio architetturale di gestione del ciclo di vita delle risorse, evitando memory leak o connessioni pendenti.

Campo	Contenuto
Input	connessione al database già avviata
Azione	<code>dbAccess.closeConnection()</code>
Atteso	l'oggetto <code>Connection</code> sottostante risulta formalmente chiuso (<code>connection.isClosed() == true</code>)

Test 3 (`testGetAvailableTablesSuccess`)

Scopo del Test: verifica l'estrazione dell'elenco delle tabelle utente tramite i metadati del database relazionale, in coerenza con il principio architetturale di incapsulamento dell'accesso ai dati fisici, escludendo viste e tabelle di sistema.

Campo	Contenuto
Input	connessione al database attiva
Azione	<code>dbAccess.getAvailableTables()</code>
Atteso	la lista restituita non è <code>null</code> , contiene almeno un elemento, include la tabella di prova <code>prova</code> , e contiene esclusivamente entità di tipo <code>TABLE</code>

Test 4 (`testGetAvailableTablesConnectionClosed`)

Scopo del Test: verifica a fronte di input non validi: la corretta propagazione degli errori SQL quando si tenta di interrogare i metadati con una sessione non più attiva, a garanzia della resilienza del modulo di persistenza.

Campo	Contenuto
Input	connessione precedentemente chiusa tramite <code>closeConnection()</code> (<code>conn == null</code> oppure <code>conn.isClosed() == true</code>)
Azione	<code>dbAccess.getAvailableTables()</code>
Atteso	viene sollevata <code>SQLException</code> con messaggio esatto "Connessione al database chiusa o non inizializzata.", senza crash non gestiti

Package: `server`

Classe Testata: `MultiServer`

Test 1 (`testServerAvvioEConnessione`)

Scopo del Test: verifica il principio architetturale di concorrenza: il server si avvia correttamente su un thread dedicato ed è in grado di accettare richieste di connessione in ingresso.

Campo	Contenuto
Input	server avviato su una porta di test, in un thread secondario
Azione	creazione di un <code>Socket</code> client e tentativo di connessione verso il server
Atteso	<code>assertTrue</code> conferma che il collegamento viene stabilito correttamente (il server è regolarmente in ascolto)

Classe Testata: `ServerOneClient`

Test 1 (`testAcquisizioneDatiOK`)

Scopo del Test: verifica l'elaborazione del comando `0` (acquisizione dati) e la corretta istanziatura di `ServerOneClient` come thread dedicato al client, in coerenza con il principio architetturale "un thread per client".

Campo	Contenuto
-------	-----------

Input	<code>ServerSocket</code> temporaneo; client che invia l'intero <code>0</code> e la stringa <code>"provaC"</code> tramite <code>ObjectOutputStream</code>
Azione	<code>new ServerOneClient(socketConnesso)</code> (avvio del thread), seguito dall'invio dei dati sopra descritti
Atteso	la stringa ricevuta in risposta corrisponde esattamente a <code>"OK"</code>

Test 2 (`testCaricamentoAlberoFallito`)

Scopo del Test: verifica a fronte di input non validi: la richiesta di caricamento (comando `2`) di un file `.dmp` inesistente viene gestita correttamente, comunicando l'errore al client in coerenza con il principio architetturale di gestione esplicita delle eccezioni.

Campo	Contenuto
Input	comando intero <code>2</code> seguito dal nome di una tabella non serializzata in precedenza
Azione	invio del comando e del nome tabella al <code>ServerOneClient</code>
Atteso	il blocco <code>catch</code> interno viene innescato; viene restituita la stringa di errore cablata <code>"Errore durante il caricamento da archivio."</code>

Test 3 (`testRecuperoTabelleOK`)

Scopo del Test: verifica l'elaborazione del comando `4` per il recupero asincrono dell'elenco delle tabelle, accertando il corretto protocollo di sincronizzazione e la serializzazione degli oggetti di risposta verso il client.

Campo	Contenuto
Input	comando intero <code>4</code> inviato tramite stream di output su socket di test
Azione	invio del comando a <code>ServerOneClient</code> , ricezione della risposta
Atteso	la prima risposta ricevuta è la stringa di sincronizzazione <code>"OK"</code> ; l'oggetto successivo è un'istanza di <code>List<String></code> , non vuota, contenente la tabella di test <code>provac</code>

Test 4 (`testRecuperoTabelleFallito`)

Scopo del Test: verifica a fronte di input non validi: la resilienza del thread di servizio e il protocollo di notifica d'errore a fronte di anomalie nel livello di persistenza durante l'esecuzione del comando 4 , garantendo la preservazione del canale socket.

Campo	Contenuto
Input	comando intero 4 accompagnato dalla simulazione di errore a livello database (es. connessione DBMS offline)
Azione	esecuzione del comando in stato di errore sul server
Atteso	il blocco catch interno viene innescato; ServerOneClient trasmette il token di fallimento ("ERROR") preservando il canale socket, invece di interrompere bruscamente il thread